

Construção de uma Biblioteca de Tabelas Hash em Python

Artur Justino

Instituição: UniSENAI Suíço-Brasileira

RESUMO

Este relatório demonstra o aprendizado do conceito de Tabelas Hash através de testes de programação e pesquisas, adicionalmente são apresentadas outras técnicas para manipulação de dados através da biblioteca Pandas. Três Datasets no formato XLSX são utilizados para as demonstrações: Transações de Clientes, Produtos e Textos.

Quatro soluções são apresentadas como Solução 1, Solução 2, Solução 3 e Solução 4.

Índice

1. Identificação do aluno e do projeto	3
2. O Problema, o Diagnóstico e Tratamento dos Dados	3
3. Uso de Tabelas Hash	10
4. Biblioteca Hash	13
5. Pitch	14

1. Identificação do aluno e do projeto

- Nome do aluno: Justino
- Curso: Inteligência e Análise de Dados
- Projeto: Aplicação de Tabelas Hash em Problemas de Dados
- Dataset Utilizado:
- Área Tecnológica: Ciência de Dados / Estrutura de Dados

2. O Problema, o Diagnóstico e Tratamento dos Dados

Dataset Produtos - Contém informações de Produtos que parecem ser de uma loja, as variáveis são ID_Produto, Nome, Categoria e Preço

1240 registros no total

6 registros únicos - variável Nome

38.6+ KB

Problemas:

Duplicados - Existem muitos valores duplicados considerando ID_Produto + Nome + Categoria + Preço, as linhas são exatamente o mesmo registro.

Inconsistência - Existem produtos com categorias incorretas

Dataset Transações dos Clientes - Contém informações de pagamentos, as variáveis são CPF, Nome, Data_Transação e Valor

1230 registros no total

6 registros únicos - variável Nome

38.4+ KB

Problemas:

Duplicados - Existem muitos valores duplicados considerando CPF + Nome + Data_Transação + Valor, as linhas são exatamente o mesmo registro

Dataset Texto - Contém frases, parecem ser de comentários

1240 registros no total

6 registros únicos - variável Nome

9.8+ KB

Problemas:

Duplicados - A maioria dos registros são duplicações. Este Dataset não possui ID, então cada registro igual se refere a mesma informação.

Efetuei comandos em Python para verificar os Datasets de maneira geral e logo já foi possível identificar inconsistências no conjunto de dados.

Com *.info()* é possível observar o tamanho do dataset, os tipos de dados e quantidade de observações e variáveis.

Com *.describe()* é possível ter uma visão geral dos dados, máximo, mínimo, médias.

Com *.nunique()* é possível identificar a quantidade de registros duplicados

Com *.isnull().values.any()* é possível identificar informações ausentes (NULLs)

Dataset Produtos

Neste conjunto de dados existem produtos com categorias incorretas, impossibilitando a análise de vendas. Exemplo: Celular com categoria de Roupa e Camisa com categoria de Eletrônico. As categorias também estão com grafia incorreta.

Imagem 1 - A imagem abaixo mostra a visão geral:

```
df_produtos.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1230 entries, 0 to 1229
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype  
---  ---
0   id_produto  1230 non-null   int64  
1   nome        1230 non-null   object  
2   categoria   1230 non-null   object  
3   preco       1230 non-null   float64 
dtypes: float64(1), int64(1), object(2)
memory usage: 38.6+ KB
```

Imagem 2 - A imagem abaixo demonstra a quantidade de itens únicos:

```
print("nome", df_produtos_clean['id_produto'].nunique())

.. nome 446
```

Imagem 3 - A imagem abaixo mostra Notebooks com categorias incorretas e a grafia em si está incorreta também

```
pd.set_option('display.max_rows', 500)
df_produtos_clean[df_produtos_clean["nome"] == "Notebook"]
```

19	157	Notebook	acessorio	3612.47
21	70	Notebook	roupa	2944.27
22	1	Notebook	roupa	2075.11
56	118	Notebook	roupa	829.02
57	498	Notebook	acessorio	1847.95
64	110	Notebook	acessorio	2959.85
67	198	Notebook	roupa	1055.79
74	324	Notebook	acessorio	433.93
77	16	Notebook	roupa	1550.51
79	477	Notebook	roupa	2032.82

Imagem 4 - A imagem abaixo mostra a categoria de Eletrônicos contendo produtos que não são eletrônicos e o texto Eletrônico grafado incorretamente

```
df_produtos_clean[df_produtos_clean["categoria"] == "eletronico"]
```

	id_produto	nome	categoria	preco
0	438	Notebook	eletronico	1218.27
2	356	Notebook	eletronico	4691.80
3	498	Relógio	eletronico	1559.90
4	7	Relógio	eletronico	2355.48
5	359	Fone	eletronico	4435.12
6	92	Camisa	eletronico	3324.89
8	313	Camisa	eletronico	1845.06
10	218	Celular	eletronico	136.37
13	193	Camisa	eletronico	627.62
16	251	Relógio	eletronico	494.94

Imagem 5 - A imagem abaixo mostra o comando de limpeza para remover duplicações:

```
df_clientes_clean = df_clientes_transacoes.drop_duplicates().copy()
df_produtos_clean = df_produtos.drop_duplicates().copy()
df_texto_clean = df_texto.drop_duplicates().copy()
```

Imagem 6 - A imagem abaixo mostra a visão geral após a limpeza:

```
df_produtos_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1200 entries, 0 to 1229
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id_produto  1200 non-null   int64
1   nome        1200 non-null   object
2   categoria   1200 non-null   object
3   preco       1200 non-null   float64
dtypes: float64(1), int64(1), object(2)
memory usage: 46.9+ KB
```

Imagem 7 - A imagem abaixo exibe a correção das categorias com .map()

```

correct_products_category = {
    "Notebook": "Eletrônico",
    "Relógio": "Acessório",
    "Fone": "Eletrônico",
    "Camisa": "Roupa",
    "Celular": "Roupa",
    "Tênis": "Acessório",
}

df_produtos_clean['categoria'] = df_produtos_clean['nome'].map(correct_products_category).fillna(df_produtos_clean['categoria'])
df_produtos_clean.sample(10)

```

	id_produto	nome	categoria	preco
620	11	Notebook	Eletrônico	1124.25
449	239	Celular	Roupa	1799.12
406	270	Celular	Roupa	4433.56
462	272	Relógio	Acessório	960.14
39	357	Camisa	Roupa	2000.03
1176	162	Camisa	Roupa	3311.12
33	353	Celular	Roupa	2656.03
872	468	Notebook	Eletrônico	1246.98

Dataset Transações dos Clientes

No Dataset de Transações dos Clientes estão presentes diversos dados duplicados. Um caso recorrente e para demonstrar é o do cliente com Nome "João", há mais de um registro referente à mesma operação, observe na imagem abaixo:

Imagem 8

```

df_clientes_transacoes[df_clientes_transacoes["nome"] == "João"]

```

	cpf	nome	data_transacao	valor
0	24637532215	João	2024-02-02	68.24
1	24637532215	João	2024-02-02	68.24

Imagem 9 - A imagem abaixo mostra a visão geral:

```

df_clientes_transacoes.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1224 entries, 0 to 1223
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   cpf              1224 non-null   int64
1   nome             1224 non-null   object
2   data_transacao   1224 non-null   datetime64[ns]
3   valor            1224 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 38.4+ KB

```

Imagem 10 - A imagem abaixo exibe como ficaria o Dataset removendo os duplicados:

```
print("cliente", df_clientes_transacoes['cpf'].nunique())
```

```
• cliente 1200
```

Imagem 11 - A imagem abaixo mostra a remoção dos itens duplicados:

```
df_clientes_clean = df_clientes_transacoes.drop_duplicates().copy()
df_produtos_clean = df_produtos.drop_duplicates().copy()
df_texto_clean = df_texto.drop_duplicates().copy()
```

Imagem 12 - A imagem abaixo mostra a visão geral do Dataset após a limpeza:

```
df_produtos_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1200 entries, 0 to 1229
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   id_produto  1200 non-null   int64
1   nome        1200 non-null   object
2   categoria   1200 non-null   object
3   preco       1200 non-null   float64
dtypes: float64(1), int64(1), object(2)
memory usage: 46.9+ KB
```


Dataset Texto

Como este dataset apresenta somente uma variável, basta remover as duplicações, neste caso não é necessária nenhuma análise e (ou) mudança além dos itens duplicados.

Imagem 13 - A imagem abaixo exibe a visão geral:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1240 entries, 0 to 1239
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   frase   1240 non-null     object
dtypes: object(1)
memory usage: 9.8+ KB
```

Imagem 14 - A imagem abaixo mostra a visão geral após a limpeza:

```
df_texto_clean.info()

<class 'pandas.core.frame.DataFrame'>
Index: 6 entries, 0 to 20
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   frase   6 non-null     object
dtypes: object(1)
memory usage: 96.0+ bytes
```

Imagem 15 - A imagem abaixo exibe uma amostra dos dados:

```
df_texto.sample(100)
```

127	entrega demorou muito
617	gostei muito do produto
559	entrega demorou muito
547	voltaria a comprar
528	gostei muito do produto
234	não gostei
1183	entrega demorou muito
582	voltaria a comprar
624	voltaria a comprar
893	entrega demorou muito
710	gostei muito do produto
17	gostei muito do produto
444	excelente qualidade
281	não gostei

3. Uso de Tabelas Hash

Solução 1

Hashing é o fato de converter uma ou mais informações em um Hash, um conjunto de letras e números de tamanho específico. Sendo assim, foi criado um algoritmo para criar um Hash a partir de todos os dados da linha, ou seja, Nome + Categoria + Preço, posteriormente os dados são adicionados em uma, onde o índice (0,0) é o Hash, o índice (0,1) é o nome, o índice (0,2) é a categoria e o último é o preço.

```
def generate_hashcode(text):
    return __builtins__.hash(str(text))

def find_product(the_list, value):
    found = []
    for i in the_list:
        if str(i[0]) == value:
            found.append(i)
    return found

products_hash = []

for index, row in df_produtos_clean.drop_duplicates(subset=['nome']).iterrows():
    product_name = row["nome"] + row["categoria"] + str(row["preco"])
    hash_code = generate_hashcode(product_name)
    product = [hash_code, product_name, row["categoria"], row["preco"]]
    products_hash.append(product)

find_product(products_hash, "-5057174383385882193")
```

```
[[-5057174383385882193, 'NotebookEletrônico1218.27', 'Eletrônico', 1218.27]]
```

Solução 2

Com algoritmos de Tabela Hash, foram implementadas funções para criar hash, adicionar e buscar itens. A Solução 1 visa somente demonstrar a lógica de Tabela Hash, sem visar a melhor arquitetura. Nas outras duas Soluções propostas é possível observar uma abordagem diferente utilizando o que a linguagem Python oferece.

Imagem 16 - Abaixo as funções de implementação dos conceitos de Tabela Hash:

```
def hash_function(value):
    value = str(value)
    sum_of_chars = 0
    for char in value:
        sum_of_chars += ord(char)
    return sum_of_chars % len(products)

def add(product_details):
    index = hash_function(product_details['nome'])
    products[index].append(product_details)

def contains(product_name):
    index = hash_function(product_name)
    for product_detail_dict in products[index]:
        if product_detail_dict['nome'] == product_name:
            return True
    return False

def get(product_name):
    index = hash_function(product_name)
    found_products = []
    for product_detail_dict in products[index]:
        if product_detail_dict['nome'] == product_name:
            found_products.append(product_detail_dict)
    return found_products if found_products else None

for i in range(len(products)):
    products[i].clear()

for index, row in df_produtos_clean.iterrows():
    product_data = {
        'nome': row['nome'],
        'categoria': row['categoria'],
        'preco': row['preco']
    }
    add(product_data)
```

Solução 3

Utilizando um dicionário, se o entendimento está correto, implicitamente já está sendo usado o conceito Hash, uma vez que não é possível duplicar ID.

Imagem 17 - Implementamos uma solução com dicionário:

```
ht_products = {}

for index, row in df_produtos_clean.iterrows():
    id_master = index
    product_details = {
        'id' : row['id_produto'],
        'name': row['nome'],
        'category': row['categoria'],
        'price': row['preco']
    }
    ht_products[id_master] = product_details

for i, a in ht_products.items():
    if a["id"] == 438:
        print(i, a)
```

Solução 4

Utilizando a biblioteca Pandas para manipular e tratar os dados. Esta biblioteca foi selecionada pela performance, flexibilidade e facilidade para trabalhar com dados.

Correção das categorias dos produtos para as categorias corretas exibida na imagem 7

Remoção de duplicados exibida na Imagem 5

Criação de arquivo .parquet para salvar o novo Dataset:

```
df_produtos_clean.to_parquet(path + "produtos.parquet", index=False)
```

4. Biblioteca Hash

Implementação de classes e funções para criar uma mini biblioteca de HashTable contendo insert, find e delete para salvar dados:

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTableLibrary:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError(">0")
        self.capacity = capacity
        self.size = 0
        self.table = [None] * capacity

    def _hash(self, key):
        if not isinstance(key, str):
            key = str(key)
        sum_of_chars = 0
        for char in key:
            sum_of_chars += ord(char)
        return sum_of_chars % self.capacity

    def add(self, key, value):
        index = self._hash(key)

        if self.table[index] is None:
            self.table[index] = Node(key, value)
            self.size += 1
        else:
            current = self.table[index]
            while current:
                if current.key == key:
                    current.value = value
                    return
                if current.next is None:
                    current.next = Node(key, value)
                    self.size += 1
                    return
                current = current.next

    def get(self, key):
```

5. Pitch

Produzi vídeos para meu podcast CodeCode falando sobre Hash:

1 - Na ideia do Pitch, em poucos minutos explicando o que é Hash.

<https://www.youtube.com/watch?v=E4CWK7hFbco>



2 - Explicando HashTable enquanto ainda aprendo e entendo o que é

https://www.youtube.com/watch?v=epD_KUBsO-g



Pitch escrito

Tabela Hash - O que é Tabela Hash

Tabela hash é um tipo de estrutura de dado na qual o endereço do índice do elemento é gerado a partir de uma função hash. Isso permite um acesso muito rápido aos dados, pois o valor do índice funciona como uma chave para o valor dos dados. Em outras palavras, as tabelas hash armazenam pares chave-valor, mas a chave é gerada por meio de uma função hash. Assim, a função de busca e inserção de um elemento de dados torna-se muito rápida, pois os próprios valores das chaves se tornam o índice do array que armazena os dados. Durante a busca, a chave é transformada em hash e o hash resultante indica onde o valor correspondente está armazenado.

O que o trabalho propõe:

Pesquisar e implementar Tabelas Hash com um dos Datasets oferecidos: Texto, Transações de Clientes e o terceiro de Produtos.

O que eu fiz:

Pesquisei o que é tabela hash, fiz muitos testes, criei um notebook Python com as implementações, um relatório técnico e um vídeo explorando o assunto.

Jingle:

Tabela Hash...

Tabela Hash...

Programar...

Lucrar até estourar...

GITHUB

<https://github.com/deveveryday/dsai-data-structure-hashtables>

REFERÊNCIAS

PFENNING, Frank; SIMMONS, Rob - Lecture 12 Hash Tables 15-122: Principles of Imperative Computation (Spring 2023)

<https://web2.qatar.cmu.edu/~mhhammou/15122-s23/lectures/13-hashing/writeup/pdf/main.pdf>

BENHAMOU, Eric; HUANG, Chien-Chung; VÁZQUEZ, Sofia- Algorithmic and advanced Programming in Python

<https://web2.qatar.cmu.edu/~mhhammou/15122-s23/lectures/13-hashing/writeup/pdf/main.pdf>

